

BANCO DE DADOS — PROJETO SESI ALIMENTAÇÃO

SISTEMA DE CANTINA ESCOLAR

1. INTRODUÇÃO

O projeto Sesi Alimentação consiste no desenvolvimento de um sistema web voltado para a realização de pedidos de alimentos e bebidas da cantina escolar. A proposta busca permitir que alunos previamente cadastrados pela instituição acessem o sistema, escolham produtos disponíveis, realizem pedidos e efetuem o pagamento de forma mais ágil e organizada.

A utilização de um banco de dados relacional foi escolhida devido à necessidade de armazenar informações estruturadas, como alunos, produtos, categorias, pedidos, itens do pedido e formas de pagamento. Esse modelo garante maior controle, consistência, integridade referencial e segurança no relacionamento entre os dados.

O Sistema Gerenciador de Banco de Dados utilizado será o MySQL, por ser uma tecnologia amplamente utilizada, confiável, compatível com aplicações web e adequada para sistemas que necessitam de organização, consultas SQL e controle de relacionamentos.

2. LEVANTAMENTO DE REQUISITOS

2.1 OBJETIVO DO BANCO DE DADOS

O banco de dados tem como objetivo armazenar e organizar as informações necessárias para o funcionamento do sistema Sesi Alimentação, permitindo o controle dos alunos, produtos cadastrados, categorias, pedidos realizados, itens comprados e formas de pagamento utilizadas.

2.2 REQUISITOS FUNCIONAIS

O banco de dados deverá permitir:

1. Armazenar os dados dos alunos previamente cadastrados pela instituição.
 2. Armazenar os dados dos administradores responsáveis pelo sistema.
 3. Cadastrar, consultar, atualizar e remover categorias de produtos.
 4. Cadastrar, consultar, atualizar e remover produtos da cantina.
 5. Registrar pedidos realizados pelos alunos.
 6. Registrar os itens pertencentes a cada pedido.
 7. Armazenar as formas de pagamento disponíveis.
 8. Consultar o histórico de pedidos dos alunos.
 9. Consultar produtos disponíveis por categoria.
 10. Consultar o valor total de cada pedido.
-

2.3 REQUISITOS NÃO FUNCIONAIS

1. O banco deverá ser relacional.
 2. O sistema deverá utilizar MySQL.
 3. As tabelas deverão possuir chaves primárias.
 4. As tabelas relacionadas deverão possuir chaves estrangeiras.
 5. Os dados deverão manter integridade referencial.
 6. O banco deverá evitar redundância de informações.
 7. A estrutura deverá estar normalizada até a Terceira Forma Normal.
 8. O banco deverá permitir consultas SQL eficientes.
 9. Campos obrigatórios deverão possuir restrição NOT NULL.
 10. Campos únicos, como e-mail, deverão utilizar restrição UNIQUE.
-

2.4 REGRAS DE NEGÓCIO

1. O aluno não realiza cadastro manual no sistema.

2. Os alunos já são previamente cadastrados pela instituição.
 3. Cada aluno pode realizar vários pedidos.
 4. Cada pedido pertence a apenas um aluno.
 5. Cada pedido pode conter um ou mais produtos.
 6. Cada produto pertence a uma categoria.
 7. Cada item do pedido representa um produto e sua quantidade.
 8. Um produto pode aparecer em vários pedidos.
 9. Cada pedido possui uma forma de pagamento.
 10. O administrador é responsável por cadastrar e gerenciar produtos e categorias.
 11. Produtos inativos não devem ser exibidos para compra.
 12. O estoque do produto deve ser controlado.
 13. O valor total do pedido deve ser calculado com base nos itens adicionados.
-

3. ENTIDADES IDENTIFICADAS

As principais entidades do banco de dados são:

1. Aluno
 2. Administrador
 3. Categoria
 4. Produto
 5. Forma de Pagamento
 6. Pedido
 7. Item do Pedido
-

4. MODELAGEM CONCEITUAL — DER

4.1 ENTIDADE ALUNO

Representa os estudantes que poderão acessar o sistema e realizar pedidos.

Atributos:

- id_aluno
- nome
- email
- senha
- turma
- status

Chave primária:

- id_aluno
-

4.2 ENTIDADE ADMINISTRADOR

Representa o usuário responsável por gerenciar produtos, categorias e acompanhar os pedidos.

Atributos:

- id_admin
- nome
- email
- senha

Chave primária:

- id_admin
-

4.3 ENTIDADE CATEGORIA

Representa os grupos de produtos vendidos na cantina.

Atributos:

- id_categoria
- nome_categoria
- descricao

Chave primária:

- id_categoria
-

4.4 ENTIDADE PRODUTO

Representa os itens disponíveis para compra na cantina.

Atributos:

- id_produto
- nome
- descricao
- preco
- estoque
- imagem
- status
- id_categoria
- id_admin

Chave primária:

- id_produto

Chaves estrangeiras:

- id_categoria
 - id_admin
-

4.5 ENTIDADE FORMA_PAGAMENTO

Representa as formas de pagamento aceitas pelo sistema.

Atributos:

- id_pagamento
- tipo_pagamento

Chave primária:

- id_pagamento
-

4.6 ENTIDADE PEDIDO

Representa o pedido realizado por um aluno.

Atributos:

- id_pedido
- data_pedido
- valor_total
- status
- id_aluno
- id_pagamento

Chave primária:

- id_pedido

Chaves estrangeiras:

- id_aluno
 - id_pagamento
-

4.7 ENTIDADE ITEM_PEDIDO

Representa os produtos inseridos dentro de um pedido.

Atributos:

- id_item
- quantidade
- valor_unitario
- subtotal
- id_pedido
- id_produto

Chave primária:

- id_item

Chaves estrangeiras:

- id_pedido
 - id_produto
-

5. RELACIONAMENTOS E CARDINALIDADES

5.1 ALUNO E PEDIDO

Um aluno pode realizar vários pedidos, mas cada pedido pertence a apenas um aluno.

Cardinalidade:

ALUNO 1:N PEDIDO

5.2 PEDIDO e ITEM_PEDIDO

Um pedido pode possuir vários itens, mas cada item pertence a apenas um pedido.

Cardinalidade:

PEDIDO 1:N ITEM_PEDIDO

5.3 PRODUTO e ITEM_PEDIDO

Um produto pode aparecer em vários itens de pedido, mas cada item do pedido se refere a apenas um produto.

Cardinalidade:

PRODUTO 1:N ITEM_PEDIDO

5.4 CATEGORIA e PRODUTO

Uma categoria pode possuir vários produtos, mas cada produto pertence a apenas uma categoria.

Cardinalidade:

CATEGORIA 1:N PRODUTO

5.5 FORMA_PAGAMENTO e PEDIDO

Uma forma de pagamento pode estar vinculada a vários pedidos, mas cada pedido possui apenas uma forma de pagamento.

Cardinalidade:

FORMA_PAGAMENTO 1:N PEDIDO

5.6 ADMINISTRADOR e PRODUTO

Um administrador pode cadastrar vários produtos, mas cada produto é cadastrado por um administrador.

Cardinalidade:

ADMINISTRADOR 1:N PRODUTO

6. MODELO LÓGICO RELACIONAL

ALUNO(id_aluno PK, nome, email, senha, turma, status)

ADMINISTRADOR(id_admin PK, nome, email, senha)

CATEGORIA(id_categoria PK, nome_categoria, descricao)

FORMA_PAGAMENTO(id_pagamento PK, tipo_pagamento)

PRODUTO(id_produto PK, nome, descricao, preco, estoque, imagem, status,
id_categoria FK, id_admin FK)

PEDIDO(id_pedido PK, data_pedido, valor_total, status, id_aluno FK, id_pagamento
FK)

ITEM_PEDIDO(id_item PK, quantidade, valor_unitario, subtotal, id_pedido FK,
id_produto FK)

7. CHAVES E INTEGRIDADE

7.1 CHAVES PRIMÁRIAS

As chaves primárias são responsáveis por identificar de forma única cada registro de uma tabela.

Chaves primárias utilizadas:

- ALUNO: id_aluno
 - ADMINISTRADOR: id_admin
 - CATEGORIA: id_categoria
 - PRODUTO: id_produto
 - FORMA_PAGAMENTO: id_pagamento
 - PEDIDO: id_pedido
 - ITEM_PEDIDO: id_item
-

7.2 CHAVES ESTRANGEIRAS

As chaves estrangeiras são utilizadas para relacionar tabelas diferentes.

Chaves estrangeiras utilizadas:

- PRODUTO.id_categoria referencia CATEGORIA.id_categoria
 - PRODUTO.id_admin referencia ADMINISTRADOR.id_admin
 - PEDIDO.id_aluno referencia ALUNO.id_aluno
 - PEDIDO.id_pagamento referencia FORMA_PAGAMENTO.id_pagamento
 - ITEM_PEDIDO.id_pedido referencia PEDIDO.id_pedido
 - ITEM_PEDIDO.id_produto referencia PRODUTO.id_produto
-

7.3 INTEGRIDADE REFERENCIAL

A integridade referencial garante que os dados relacionados entre as tabelas permaneçam consistentes. Por exemplo, um pedido não poderá ser registrado para um aluno inexistente, e um item de pedido não poderá fazer referência a um produto que não existe.

8. NORMALIZAÇÃO

8.1 PRIMEIRA FORMA NORMAL — 1FN

A Primeira Forma Normal exige que todos os atributos sejam atômicos, ou seja, que cada campo armazene apenas uma informação.

No banco do projeto, os dados foram separados em campos individuais. Por exemplo, na tabela ALUNO, o nome, e-mail, senha, turma e status estão em campos separados.

Dessa forma, o banco atende à 1FN.

8.2 SEGUNDA FORMA NORMAL — 2FN

A Segunda Forma Normal exige que todos os atributos dependam totalmente da chave primária da tabela.

No projeto, cada tabela possui uma chave primária própria, e os atributos dependem diretamente dessa chave. Por exemplo, na tabela PRODUTO, os campos nome, descrição, preço, estoque, imagem e status dependem diretamente do id_produto.

Dessa forma, o banco atende à 2FN.

8.3 TERCEIRA FORMA NORMAL — 3FN

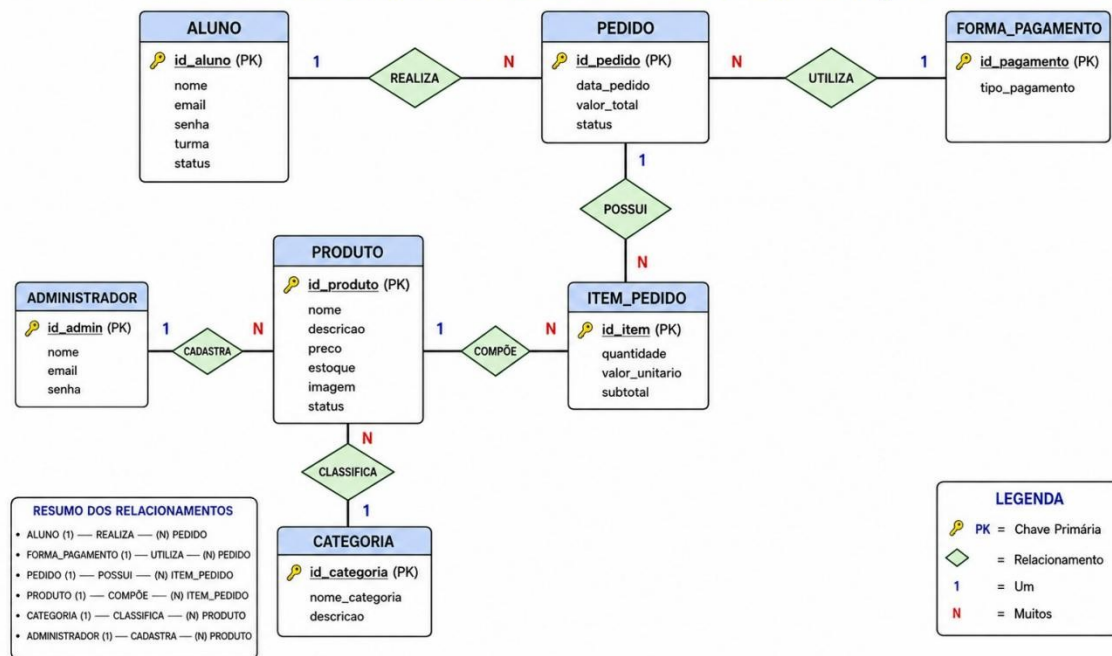
A Terceira Forma Normal exige que não existam dependências transitivas, ou seja, um atributo não chave não deve depender de outro atributo não chave.

No projeto, dados como categoria e forma de pagamento foram separados em tabelas próprias. Assim, o nome da categoria não é repetido dentro da tabela PRODUTO; em vez disso, utiliza-se uma chave estrangeira.

Isso reduz redundância, melhora a organização e facilita a manutenção do banco.

Dessa forma, o banco atende à 3FN.

DIAGRAMA CONCEITUAL – SESI ALIMENTAÇÃO



9. IMPLEMENTAÇÃO FÍSICA — SCRIPT SQL MYSQL

```
CREATE DATABASE sesi_alimentacao;
```

```
USE sesi_alimentacao;
```

```
CREATE TABLE aluno (
    id_aluno INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL,
    turma VARCHAR(50) NOT NULL,
    status ENUM('ATIVO', 'INATIVO') NOT NULL DEFAULT 'ATIVO'
);
```

```
CREATE TABLE administrador (
```

```
id_admin INT AUTO_INCREMENT PRIMARY KEY,  
nome VARCHAR(100) NOT NULL,  
email VARCHAR(100) NOT NULL UNIQUE,  
senha VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE categoria (  
id_categoria INT AUTO_INCREMENT PRIMARY KEY,  
nome_categoria VARCHAR(80) NOT NULL UNIQUE,  
descricao VARCHAR(255)  
);
```

```
CREATE TABLE forma_pagamento (  
id_pagamento INT AUTO_INCREMENT PRIMARY KEY,  
tipo_pagamento VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE produto (  
id_produto INT AUTO_INCREMENT PRIMARY KEY,  
nome VARCHAR(100) NOT NULL,  
descricao VARCHAR(255),  
preco DECIMAL(10,2) NOT NULL,  
estoque INT NOT NULL DEFAULT 0,  
imagem VARCHAR(255),  
status ENUM('ATIVO', 'INATIVO') NOT NULL DEFAULT 'ATIVO',  
id_categoria INT NOT NULL,  
id_admin INT NOT NULL,
```

```
CONSTRAINT fk_produto_categoria  
FOREIGN KEY (id_categoria)  
REFERENCES categoria(id_categoria),
```

CONSTRAINT fk_produto_admin
FOREIGN KEY (id_admin)
REFERENCES administrador(id_admin),

CONSTRAINT chk_produto_preco
CHECK (preco >= 0),

CONSTRAINT chk_produto_estoque
CHECK (estoque >= 0)

);

CREATE TABLE pedido (
 id_pedido INT AUTO_INCREMENT **PRIMARY KEY**,
 data_pedido DATETIME **NOT NULL DEFAULT** CURRENT_TIMESTAMP,
 valor_total DECIMAL(10,2) **NOT NULL DEFAULT** 0,
 status ENUM('PENDENTE', 'PAGO', 'EM_PREPARO', 'PRONTO', 'ENTREGUE',
'CANCELADO') **NOT NULL DEFAULT** 'PENDENTE',
 id_aluno INT **NOT NULL**,
 id_pagamento INT **NOT NULL**,

CONSTRAINT fk_pedido_aluno
FOREIGN KEY (id_aluno)
REFERENCES aluno(id_aluno),

CONSTRAINT fk_pedido_pagamento
FOREIGN KEY (id_pagamento)
REFERENCES forma_pagamento(id_pagamento),

CONSTRAINT chk_pedido_valor_total
CHECK (valor_total >= 0)

);

```
CREATE TABLE item_pedido (  
    id_item INT AUTO_INCREMENT PRIMARY KEY,  
    quantidade INT NOT NULL,  
    valor_unitario DECIMAL(10,2) NOT NULL,  
    subtotal DECIMAL(10,2) NOT NULL,  
    id_pedido INT NOT NULL,  
    id_produto INT NOT NULL,
```

```
CONSTRAINT fk_item_pedido  
FOREIGN KEY (id_pedido)  
REFERENCES pedido(id_pedido)  
ON DELETE CASCADE,
```

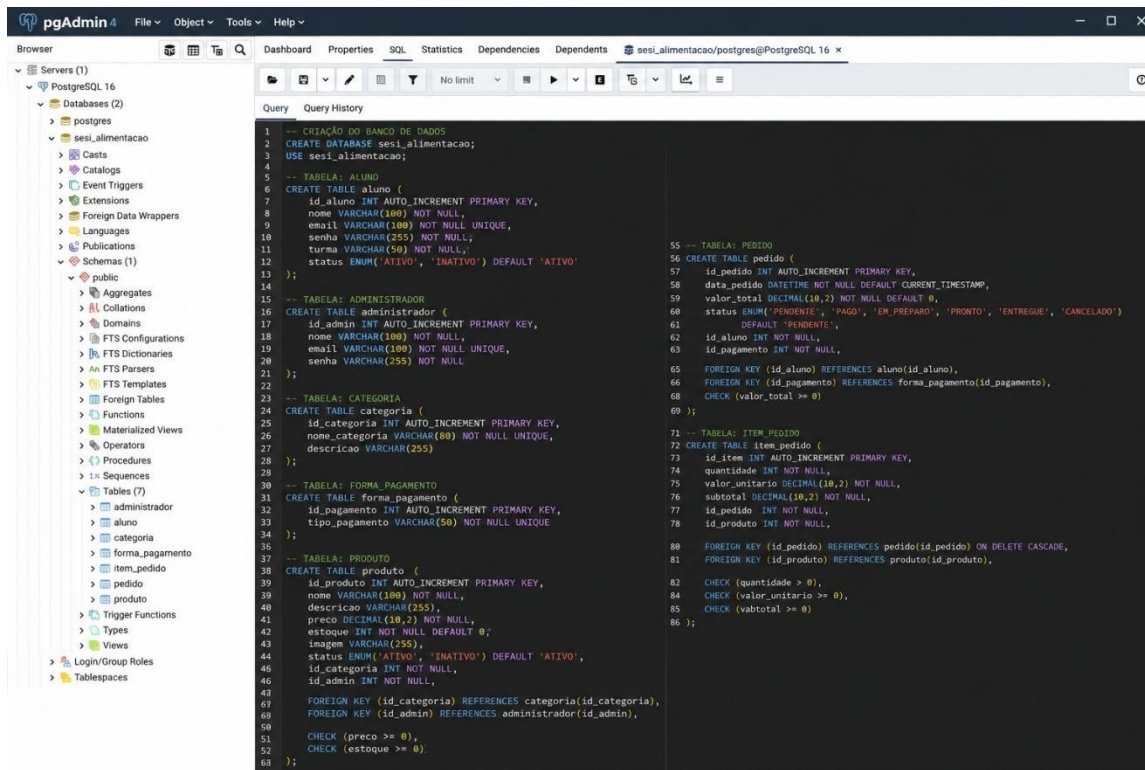
```
CONSTRAINT fk_item_produto  
FOREIGN KEY (id_produto)  
REFERENCES produto(id_produto),
```

```
CONSTRAINT chk_item_quantidade  
CHECK (quantidade > 0),
```

```
CONSTRAINT chk_item_valor_unitario  
CHECK (valor_unitario >= 0),
```

```
CONSTRAINT chk_item_subtotal  
CHECK (subtotal >= 0)
```

);



10. INSERÇÃO DE DADOS — DML

INSERT INTO aluno (nome, email, senha, turma, status)

VALUES

('João Pedro Santos', 'joao.pedro@senai.edu.br', '123456', '2DS', 'ATIVO'),
('Maria Clara Souza', 'maria.clara@senai.edu.br', '123456', '2DS', 'ATIVO');

INSERT INTO administrador (nome, email, senha)

VALUES

('Administrador SESI', 'admin@sesi.com.br', 'admin123');

INSERT INTO categoria (nome_categoria, descricao)

VALUES

('Salgados', 'Produtos salgados vendidos na cantina'),
('Bebidas', 'Bebidas disponíveis para compra'),

('Doces', 'Produtos doces vendidos na cantina');

INSERT INTO forma_pagamento (tipo_pagamento)

VALUES

('Pix'),

('Cartão de Débito'),

('Cartão de Crédito'),

('Dinheiro');

INSERT INTO produto (nome, descricao, preco, estoque, imagem, status, id_categoria, id_admin)

VALUES

('Coxinha', 'Salgado de frango', 6.50, 30, 'coxinha.png', 'ATIVO', 1, 1),

('Suco Natural', 'Suco de laranja 300ml', 5.00, 20, 'suco.png', 'ATIVO', 2, 1),

('Brigadeiro', 'Doce de chocolate', 3.00, 40, 'brigadeiro.png', 'ATIVO', 3, 1);

INSERT INTO pedido (valor_total, status, id_aluno, id_pagamento)

VALUES

(11.50, 'PAGO', 1, 1);

INSERT INTO item_pedido (quantidade, valor_unitario, subtotal, id_pedido, id_produto)

VALUES

(1, 6.50, 6.50, 1, 1),

(1, 5.00, 5.00, 1, 2);

11. Consultas SQL — DQL

11.1 Consultar todos os alunos

SELECT * FROM aluno;

11.2 CONSULTAR PRODUTOS ATIVOS

SELECT

p.id_produto,
p.nome,
p.descricao,
p.preco,
p.estoque,
c.nome_categoria

FROM produto p

INNER JOIN categoria c **ON** p.id_categoria = c.id_categoria

WHERE p.status = 'ATIVO';

11.3 CONSULTAR PEDIDOS DE UM ALUNO

SELECT

pe.id_pedido,
a.nome **AS** aluno,
pe.data_pedido,
pe.valor_total,
pe.status,
fp.tipo_pagamento

FROM pedido pe

INNER JOIN aluno a **ON** pe.id_aluno = a.id_aluno

INNER JOIN forma_pagamento fp **ON** pe.id_pagamento = fp.id_pagamento

WHERE a.id_aluno = 1;

11.4 Consultar itens de um pedido

SELECT

pe.id_pedido,
pr.nome **AS** produto,
ip.quantidade,
ip.valor_unitario,
ip.subtotal

```
FROM item_pedido ip
INNER JOIN pedido pe ON ip.id_pedido = pe.id_pedido
INNER JOIN produto pr ON ip.id_produto = pr.id_produto
WHERE pe.id_pedido = 1;
```

11.5 CONSULTAR TOTAL VENDIDO POR PRODUTO

SELECT

```
    pr.nome AS produto,
    SUM(ip.quantidade) AS quantidade_vendida,
    SUM(ip.subtotal) AS total_arrecadado
```

```
FROM item_pedido ip
INNER JOIN produto pr ON ip.id_produto = pr.id_produto
GROUP BY pr.nome
ORDER BY quantidade_vendida DESC;
```

12. ATUALIZAÇÃO DE DADOS — UPDATE

12.1 Atualizar preço de produto

```
UPDATE produto
SET preco = 7.00
WHERE id_produto = 1;
```

12.2 Atualizar status de pedido

```
UPDATE pedido
SET status = 'EM_PREPARO'
WHERE id_pedido = 1;
```

12.3 Inativar produto

```
UPDATE produto
SET status = 'INATIVO'
WHERE id_produto = 3;
```

13. EXCLUSÃO DE DADOS — DELETE

13.1 Excluir categoria

```
DELETE FROM categoria  
WHERE id_categoria = 3;
```

Observação: a exclusão só será permitida se não houver produtos vinculados a essa categoria.

13.2 Excluir pedido

```
DELETE FROM pedido  
WHERE id_pedido = 1;
```

Como a tabela item_pedido utiliza ON DELETE CASCADE, os itens relacionados ao pedido também serão removidos automaticamente.

14. RECURSOS AVANÇADOS

14.1 VIEW

Embora o uso de recursos avançados não tenha sido obrigatório, foi criada uma View para facilitar a consulta de pedidos completos.

```
CREATE VIEW vw_pedidos_completos AS
```

```
SELECT  
pe.id_pedido,  
a.nome AS aluno,  
a.turma,  
pe.data_pedido,  
pe.status AS status_pedido,  
fp.tipo_pagamento,  
pr.nome AS produto,  
ip.quantidade,
```

```
ip.valor_unitario,  
ip.subtotal,  
pe.valor_total  
FROM pedido pe  
INNER JOIN aluno a ON pe.id_aluno = a.id_aluno  
INNER JOIN forma_pagamento fp ON pe.id_pagamento = fp.id_pagamento  
INNER JOIN item_pedido ip ON pe.id_pedido = ip.id_pedido  
INNER JOIN produto pr ON ip.id_produto = pr.id_produto;
```

Consulta da View:

```
SELECT * FROM vw_pedidos_completos;
```

14.2 PROCEDURE – LISTAR PRODUTOS ATIVOS

Para facilitar consultas recorrentes do sistema, foi criada uma Procedure responsável por listar apenas os produtos disponíveis para compra.

```
DELIMITER //
```

```
CREATE PROCEDURE ListarProdutosAtivos()  
BEGIN  
SELECT *  
FROM produto  
WHERE status = 'ATIVO';  
END //
```

```
DELIMITER ;
```

Execução:

```
CALL ListarProdutosAtivos();
```

14.3 PROCEDURE – CONSULTAR PEDIDOS DE UM ALUNO

Foi desenvolvida uma Procedure para consultar todos os pedidos realizados por um aluno específico.

```
DELIMITER //
```

```
CREATE PROCEDURE BuscarPedidosAluno(IN p_id_aluno INT)  
BEGIN  
SELECT  
id_pedido,  
data_pedido,  
valor_total,
```

```
status
FROM pedido
WHERE id_aluno = p_id_aluno;
END //
```

DELIMITER ;

Execução:

```
CALL BuscarPedidosAluno(1);
```

15. Dicionário de Dados

15.1 Tabela ALUNO

Campo	Tipo	Obrigatório	Descrição
id_aluno	INT	Sim	Identificador único do aluno
nome	VARCHAR(100)	Sim	Nome completo do aluno
email	VARCHAR(100)	Sim	E-mail institucional do aluno
senha	VARCHAR(255)	Sim	Senha de acesso do aluno
turma	VARCHAR(50)	Sim	Turma do aluno
status	ENUM	Sim	Situação do aluno no sistema

15.2 Tabela ADMINISTRADOR

Campo	Tipo	Obrigatório	Descrição
id_admin	INT	Sim	Identificador único do administrador
nome	VARCHAR(100)	Sim	Nome do administrador
email	VARCHAR(100)	Sim	E-mail do administrador
senha	VARCHAR(255)	Sim	Senha de acesso do administrador

15.3 Tabela CATEGORIA

Campo	Tipo	Obrigatório	Descrição
id_categoria	INT	Sim	Identificador único da categoria
nome_categoria	VARCHAR(80)	Sim	Nome da categoria
descricao	VARCHAR(255)	Não	Descrição da categoria

15.4 Tabela FORMA_PAGAMENTO

Campo	Tipo	Obrigatório	Descrição
id_pagamento	INT	Sim	Identificador único da forma de pagamento
tipo_pagamento	VARCHAR(50)	Sim	Tipo de pagamento aceito

15.5 Tabela PRODUTO

Campo	Tipo	Obrigatório	Descrição
id_produto	INT	Sim	Identificador único do produto
nome	VARCHAR(100)	Sim	Nome do produto
descricao	VARCHAR(255)	Não	Descrição do produto
preco	DECIMAL(10,2)	Sim	Preço do produto
estoque	INT	Sim	Quantidade disponível em estoque
imagem	VARCHAR(255)	Não	Caminho ou nome da imagem do produto
status	ENUM	Sim	Situação do produto
id_categoria	INT	Sim	Categoria do produto

Campo	Tipo	Obrigatório	Descrição
id_admin	INT	Sim	Administrador responsável pelo cadastro

15.6 Tabela PEDIDO

Campo	Tipo	Obrigatório	Descrição
id_pedido	INT	Sim	Identificador único do pedido
data_pedido	DATETIME	Sim	Data e horário em que o pedido foi realizado
valor_total	DECIMAL(10,2)	Sim	Valor total do pedido
status	ENUM	Sim	Situação atual do pedido
id_aluno	INT	Sim	Aluno que realizou o pedido
id_pagamento	INT	Sim	Forma de pagamento utilizada

15.7 Tabela ITEM_PEDIDO

Campo	Tipo	Obrigatório	Descrição
id_item	INT	Sim	Identificador único do item
quantidade	INT	Sim	Quantidade

Campo	Tipo	Obrigatório	Descrição
			comprada do produto
valor_unitario	DECIMAL(10,2)	Sim	Valor do produto no momento da compra
subtotal	DECIMAL(10,2)	Sim	Resultado de quantidade vezes valor unitário
id_pedido	INT	Sim	Pedido ao qual o item pertence
id_produto	INT	Sim	Produto comprado

16. JUSTIFICATIVA DA ESCOLHA DO BANCO RELACIONAL

O banco de dados relacional foi escolhido porque o sistema Sesi Alimentação possui informações bem estruturadas e fortemente relacionadas entre si. Alunos realizam pedidos, pedidos possuem itens, itens estão ligados a produtos, produtos pertencem a categorias e pedidos possuem formas de pagamento.

Esse tipo de organização exige controle de integridade, evitando que sejam cadastrados pedidos sem alunos, produtos sem categoria ou itens sem pedido. O MySQL permite utilizar chaves primárias, chaves estrangeiras, restrições e consultas SQL, garantindo maior confiabilidade e organização.

Portanto, o modelo relacional é o mais adequado para o projeto, pois atende às necessidades de consistência, segurança, organização e controle dos dados.

17. CONCLUSÃO

A modelagem do banco de dados do projeto Sesi Alimentação foi desenvolvida com base nas necessidades do sistema e nos critérios de avaliação da disciplina de Banco de Dados. Foram identificadas as principais entidades, seus atributos, relacionamentos, cardinalidades, chaves primárias e estrangeiras.

Além disso, o modelo foi normalizado até a Terceira Forma Normal, reduzindo redundâncias e garantindo maior organização dos dados. A implementação física foi elaborada em MySQL, utilizando comandos DDL, DML e DQL. Além disso, a implementação utiliza comandos DDL, DML e DQL, bem como recursos avançados do MySQL, incluindo uma View e Procedures para otimizar consultas e operações do sistema.

Dessa forma, o banco de dados proposto atende aos requisitos do sistema e contribui para o funcionamento eficiente da aplicação web Sesi Alimentação.